

DAA PRACTICAL ASSIGNMENT

Assignment 2: Sorting Algorithms—Time complexity $O(n^2)$.

Name: Mohd Sami

RollNum: 00916449825

Course: BTech CSE-DS

Github: <https://github.com/Mohd-Sami010/Cpp>

1. ASSIGNMENT STATEMENT

Write a program implementing three different sorting algorithms having a time complexity of $O(n^2)$.

2. THEORY

Bubble Sort repeatedly compares adjacent elements and swaps them if they are out of order, letting larger elements "bubble up" to their correct position over multiple passes; its nested-loop structure gives it $O(n^2)$ time complexity in the worst and average case.

Selection Sort repeatedly finds the minimum element from the unsorted portion of the array and places it at the beginning; since it scans the remaining unsorted elements for every position, it also has $O(n^2)$ time complexity.

Insertion Sort builds the sorted array one element at a time by taking each element and inserting it into its correct position among the already sorted elements; in the worst case (reverse-sorted input), each insertion requires shifting many elements, resulting in $O(n^2)$ time complexity.

3. CODE

```
#include <iostream>
#include <ctime>
#include <cstdlib>
using namespace std;
void BubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```

    }
}
}
}
void SelectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
                minIndex = j;
        }
        int temp = arr[minIndex];
        arr[minIndex] = arr[i];
        arr[i] = temp;
    }
}
void InsertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
void PrintArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
int main()
{
    int n;
    cout << "Enter array size: ";
    cin >> n;
    srand(time(0));
    int *original = new int[n];
    for (int i = 0; i < n; i++)
        original[i] = rand() % 10000;
    int *arr1 = new int[n];
    int *arr2 = new int[n];
    int *arr3 = new int[n];
    for (int i = 0; i < n; i++)
    {

```

```

        arr1[i] = original[i];
        arr2[i] = original[i];
        arr3[i] = original[i];
    }
    if (n <= 20)
    {
        cout << "\nOriginal Array:\n";
        PrintArray(original, n);
    }
    clock_t start1 = clock();
    BubbleSort(arr1, n);
    clock_t end1 = clock();
    clock_t ticks1 = end1 - start1;
    clock_t start2 = clock();
    SelectionSort(arr2, n);
    clock_t end2 = clock();
    clock_t ticks2 = end2 - start2;
    clock_t start3 = clock();
    InsertionSort(arr3, n);
    clock_t end3 = clock();
    clock_t ticks3 = end3 - start3;
    if (n <= 20)
    {
        cout << "\nBubble Sort:\n";
        PrintArray(arr1, n);
        cout << "\nSelection Sort:\n";
        PrintArray(arr2, n);
        cout << "\nInsertion Sort:\n";
        PrintArray(arr3, n);
    }
    cout << "\nAlgorithm      Clock Ticks\n";
    cout << "-----\n";
    cout << "Type: Bubble Sort" << endl;
    cout << "Ticks: " << ticks1 << endl
        << endl;
    cout << "Type: Selection Sort" << endl;
    cout << "Ticks: " << ticks2 << endl
        << endl;
    cout << "Type: Insertion Sort" << endl;
    cout << "Ticks: " << ticks3 << endl
        << endl;
    return 0;
}

```

4. OUTPUT

```
Enter array size: 10
Original Array:
9632 3065 2541 2275 2686 106 6880 4252 9342 8600

Bubble Sort:
106 2275 2541 2686 3065 4252 6880 8600 9342 9632

Selection Sort:
106 2275 2541 2686 3065 4252 6880 8600 9342 9632

Insertion Sort:
106 2275 2541 2686 3065 4252 6880 8600 9342 9632

Algorithm■■■Clock Ticks
-----
Type: Bubble Sort
Ticks: 0

Type: Selection Sort
Ticks: 0

Type: Insertion Sort
Ticks: 0
```

```
Enter array size: 100
Algorithm■■■Clock Ticks
-----
Type: Bubble Sort
Ticks: 0

Type: Selection Sort
Ticks: 0

Type: Insertion Sort
Ticks: 0
```

Enter array size: 1000

Algorithm■■Clock Ticks

Type: Bubble Sort

Ticks: 13

Type: Selection Sort

Ticks: 2

Type: Insertion Sort

Ticks: 1

Enter array size: 10000

Algorithm■■Clock Ticks

Type: Bubble Sort

Ticks: 340

Type: Selection Sort

Ticks: 155

Type: Insertion Sort

Ticks: 88

Enter array size: 30000

Algorithm■■Clock Ticks

Type: Bubble Sort

Ticks: 3300

Type: Selection Sort

Ticks: 1410

Type: Insertion Sort

Ticks: 839

5. COMPLEXITY ANALYSIS

Time Complexity: $O(n^2)$

But Insertion sort can behave $O(n)$ if array is sorted.

Space Complexity: $O(n^2)$

Conclusion: They SUCKS!!

Don't use these ones.

6. PERFORMANCE RESULTS

Data Size	Bubble Sort	Selection Sort	Insertion Sort
10	0.00	0.00	0.00
100	0.00	0.00	0.00
1000	5.67	0.67	0.33
10000	339.67	150.33	99.00
30000	3407.67	1362.67	789.00

7. GRAPHICAL ANALYSIS

